EJERCICIO PRÁCTICO · ARQUITECTURA DE SOLUCIONES

Banca por Internet BP
Diseño de arquitectura con modelo C4

Consulta de movimientos, transferencias y pagos propios e interbancarios, con onboarding biométrico, OAuth 2.0, auditoría, alta disponibilidad y recuperación ante desastres sobre AWS.

Autor: Diego Gerardo Zamora Cruz
Rol: Arquitecto de soluciones
Fecha: octubre de 2026
Versión: 1.0

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Contenido

Ilustraciones 3
Tablas 3
1. Resumen de la solución 4
2. Supuestos y alcance 4
3. Nivel 1: Diagrama de contexto 5
4. Nivel 2: Diagrama de contenedores 6
4.1. Contenedores y responsabilidades 6
4.2. Servicios agregados 7
5. Nivel 3: Diagramas de componentes 9
5.1. Backend: API Gateway, microservicios e integración 9
5.2. Patrones usados en el backend 10
5.3. Autenticación y onboarding biométrico 11
6. Aplicaciones front-end 12
7. Autenticación y autorización 12
7.1. Configuración por canal 13
7.2. Métodos de autenticación 13
7.3. Autorización en el backend 14
8. Onboarding biométrico 14
8.1. Flujo 14
8.2. Tratamiento de los datos biométricos 15
9. Capa de integración y microservicios 15
9.1. Comunicación entre componentes 15
9.2. Transferencias interbancarias 16
10. Clientes frecuentes y auditoría 16
10.1. Persistencia para clientes frecuentes 16
10.2. Auditoría 17
11. Notificaciones 17
12. Infraestructura en AWS 18
12.1. Alta disponibilidad y tolerancia a fallos 18
12.2. Recuperación ante desastres 19
12.3. Auto-healing 19
12.4. Región y latencia 19
13. Seguridad 20
14. Monitoreo y excelencia operativa 20
14.1. Observabilidad 20
14.2. Objetivos de servicio y alertas 20
15. Marco normativo y estándares 21
16. Manejo de costos 21
17. Registro de decisiones de arquitectura 22
17.1. Riesgos y mitigaciones 23
Anexo A. Diagramas de secuencia 24

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

A.1 Login en la SPA con Authorization Code + PKCE 24
A.2 Onboarding con reconocimiento facial 24
A.3 Transferencia interbancaria 25
Anexo B. Fuentes de los diagramas 25
Referencias 25

Ilustraciones

Ilustración 1: Diagrama contexto 5
Ilustración 2: Diagrama de contenedores 6
Ilustración 3: Diagramas de componentes 9
Ilustración 4: Componentes de autenticación y onboarding 11
Ilustración 5: Infraestructura en AWS 18
Ilustración 6: Login en la SPA 24
Ilustración 7: Onboarding biométrico 24
Ilustración 8: Transferencia interbancaria con saga y outbox 25

Tablas

Tabla 1: Detalle de Diagrama de contexto 5
Tabla 2: Detalle de contenedores y responsabilidades 7
Tabla 3: Decisión: microservicios por capacidad de negocio 8
Tabla 4: Patrones usados en el backend 10
Tabla 5: React con TypeScript para la SPA 12
Tabla 6: Flutter para la app móvil 12
Tabla 7: Authorization Code con PKCE en ambos canales 13
Tabla 8: Configuración por canal 13
Tabla 9: BFF Web guarda los tokens de la SPA 13
Tabla 10: Proveedor especializado de biometría 14
Tabla 11: Amazon API Gateway como puerta de la capa de integración 15
Tabla 12: Amazon ECS sobre Fargate para correr los contenedores 15
Tabla 13: Spring Boot (Java) para los BFF y los microservicios 15
Tabla 14: Comunicación entre componentes 16
Tabla 15: Comunicación asíncrona con EventBridge + SQS 16
Tabla 16: Persistencia para clientes frecuentes 16
Tabla 17: Amazon ElastiCache (Redis) como caché distribuida 17
Tabla 18: DynamoDB + S3 Object Lock para la auditoría 17
Tabla 19: Notificaciones 17
Tabla 20:Notificaciones como servicio propio que consume eventos 18
Tabla 21: Alta disponibilidad y tolerancia a fallos 19
Tabla 22: Warm standby en una segunda región 19
Tabla 23: Seguridad 20
Tabla 24: Objetivos de servicio y alertas 21
Tabla 25: Marco normativo y estándares 21
Tabla 26: Manejo de costos 22
Tabla 27: Registro de decisiones de arquitectura 23
Tabla 28: Riesgos y mitigaciones 23

1. Resumen de la solución

BP necesita una banca por internet donde sus clientes vean el histórico de movimientos, transfieran entre cuentas propias, a terceros de BP y a otros bancos, y paguen servicios. La información del cliente está en dos sistemas: el Core bancario (datos básicos, productos y movimientos) y un sistema complementario con el detalle. La norma obliga a notificar cada movimiento, y los clientes nuevos se registran desde el celular con reconocimiento facial.

La propuesta se fundamenta en cinco ideas:

- **Dos canales, un mismo backend.** Una SPA en React y una app en Flutter. Cada una tiene su propio BFF (Backend for Frontend), así la web y el móvil pueden cambiar sin afectarse entre sí.
- **Identidad delegada.** El producto OAuth 2.0 / OpenID Connect que ya tiene BP se encarga del login. Recomiendo Authorization Code con PKCE en ambos canales. En la web, los tokens quedan en el BFF y el navegador solo recibe una cookie.
- **Microservicios por capacidad de negocio** detrás de un API Gateway: Clientes, Movimientos, Transferencias, Pagos, Onboarding, Notificaciones y Auditoría. Los sistemas existentes se consumen a través de adaptadores con circuit breaker.
- **Eventos para lo que no tiene que ser inmediato.** Las transferencias publican un evento cuando terminan. Notificaciones y Auditoría lo consumen por separado, así un proveedor de SMS caído no detiene una transferencia.
- **AWS en dos regiones.** Tres zonas de disponibilidad en la región principal y una región de recuperación en warm standby, conectadas al centro de datos de BP por dos enlaces Direct Connect y una VPN IPSec de respaldo.

2. Supuestos y alcance

- BP opera en Ecuador. Por eso el marco normativo cita la Superintendencia de Bancos, la LOPDP y el SPI del Banco Central.
- El Core y el sistema complementario siguen en el centro de datos de BP y exponen servicios REST o SOAP.
- El proveedor de identidad de BP soporta OAuth 2.0, OpenID Connect, PKCE, MFA y passkeys (FIDO2).
- El enunciado no da volúmenes de clientes ni de transacciones. Por eso no fijo tamaños de servidores ni montos en dólares. Explico cómo escala cada pieza y cómo se dimensionaría con los datos reales de BP.
- Elijo AWS como nube. En la sección 12 dejo las equivalencias en Azure.
- El alcance llega hasta el nivel de componentes del modelo C4, más la infraestructura y tres diagramas de secuencia.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

3. Nivel 1: Diagrama de contexto

Presentación para usuarios no técnicos. Muestra quién usa el sistema y con qué sistemas de BP y de terceros se relacionan.

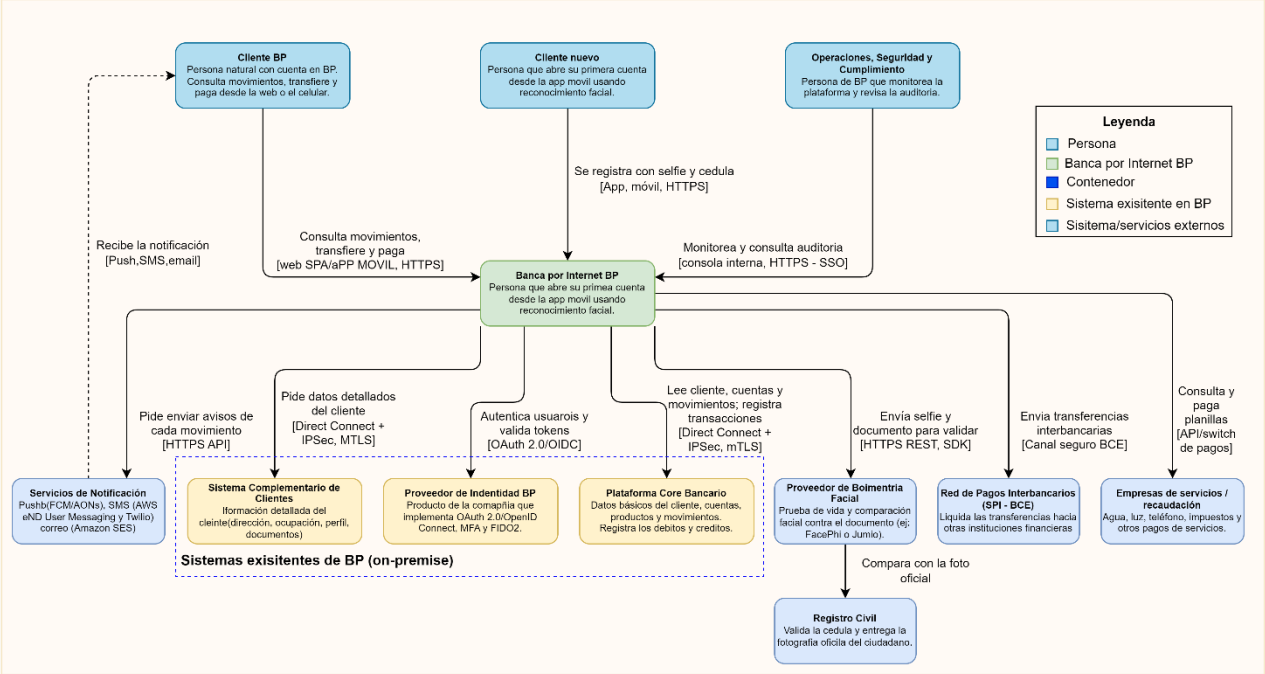


Ilustración 1: Diagrama contexto

Actor o sistema	Qué hace en la solución
Cliente BP	Entra por la web o la app para ver saldos y movimientos, transferir y pagar.
Cliente nuevo	Abre su cuenta desde la app con una selfie y su cédula, sin ir a una agencia.
Operaciones, Seguridad y Cumplimiento	Vigilan que la plataforma funcione y consultan la auditoría cuando hay un reclamo, una investigación o una revisión del regulador.
Plataforma Core Bancario	Fuente oficial de clientes, cuentas, saldos y movimientos. Ahí se registran el débito y el crédito de cada transacción.
Sistema Complementario	Guarda el detalle del cliente (dirección, actividad económica, documentos). Solo se consulta cuando una pantalla lo necesita.
Proveedor de Identidad BP	Producto existente que valida usuario y clave o passkey, y emite los tokens OAuth 2.0.
Proveedor de biometría y Registro Civil	Confirman que la persona frente a la cámara está viva y que es la dueña de la cédula.
SPI del Banco Central	Liquida las transferencias hacia otros bancos.
Empresas de servicios	Agua, luz, teléfono, impuestos. Reciben los pagos de servicios.
Servicios de notificación	Llevan el aviso de cada movimiento al cliente por push, SMS o correo.

Tabla 1: Detalle de Diagrama de contexto

Nota: La conexión con los sistemas del centro de datos de BP va por un enlace privado (AWS Direct Connect) con cifrado. El tráfico entre la nube y el Core no pasa por internet público.

4. Nivel 2: Diagrama de contenedores

Punto de vista técnico. Aplicaciones, servicios, bases de datos y mensajería, con los componentes de nube a alto nivel.

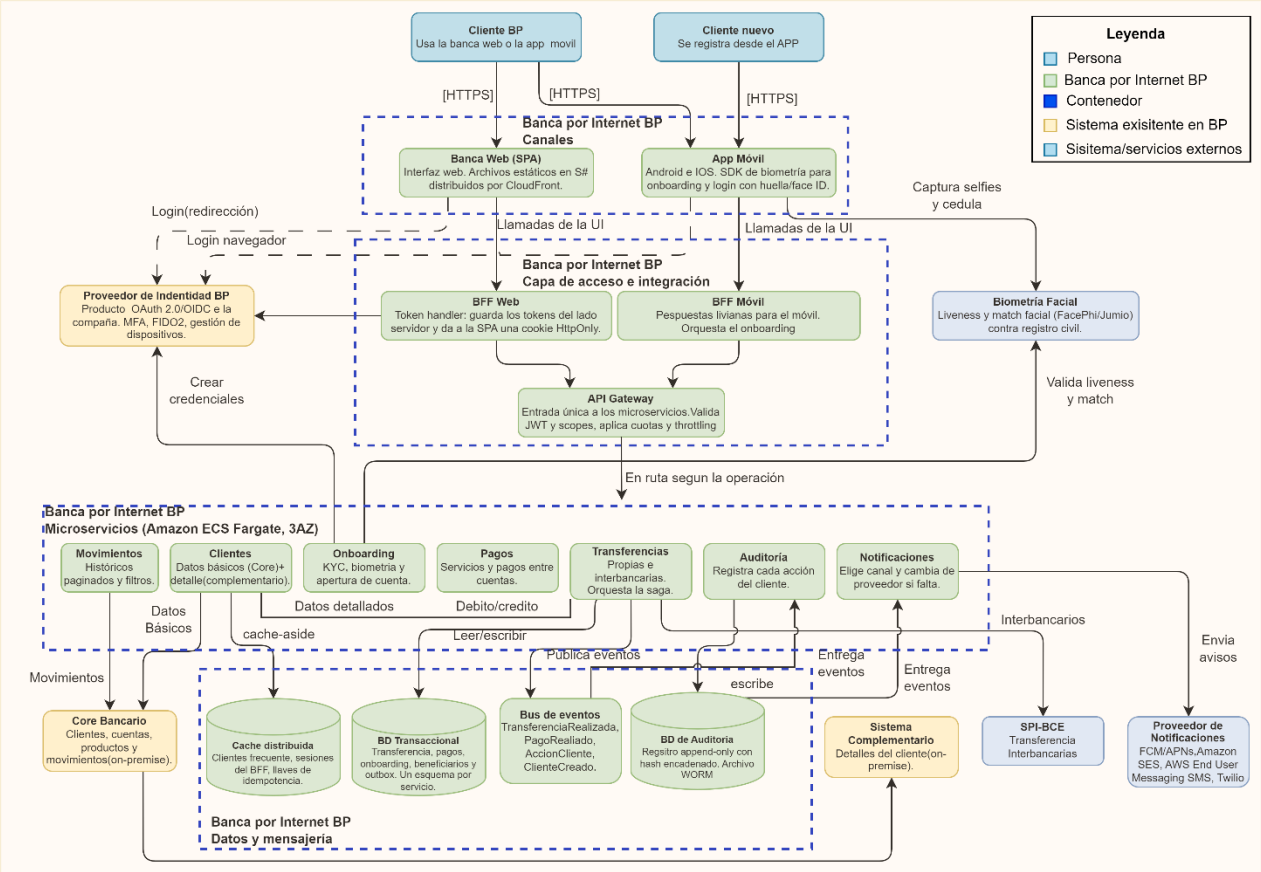


Ilustración 2: Diagrama de contenedores

4.1. Contenedores y responsabilidades

Contenedor	Tecnología	Responsabilidad
Banca Web (SPA)	React + TypeScript, servida desde S3 y CloudFront	Interfaz web. No guarda tokens, solo una cookie de sesión HttpOnly.
App Móvil	Flutter	Android e iOS con un solo código. Incluye el SDK de biometría y el login con huella o Face ID.
BFF Web	Spring Boot en ECS Fargate	Hace el login OAuth en nombre de la SPA y guarda los tokens. Arma las respuestas para cada pantalla.
BFF Móvil	Spring Boot en ECS Fargate	Respuestas livianas para redes móviles y orquestación del onboarding.
API Gateway	Amazon API Gateway + AWS WAF	Puerta única hacia los microservicios: valida el token, aplica cuotas y versiona las APIs.
Clientes	Spring Boot	Datos básicos desde el Core y detalle desde el sistema complementario, con caché para clientes frecuentes.
Movimientos	Spring Boot	Histórico paginado con filtros por fecha, monto y tipo.
Transferencias	Spring Boot	Propias, a terceros BP e interbancarias. Evita duplicados, aplica límites y reversa si algo falla.
Pagos	Spring Boot	Pago de servicios. Sigue el mismo diseño que Transferencias.

Contenedor	Tecnología	Responsabilidad
Onboarding	Spring Boot	Registro del cliente nuevo: biometría, listas restrictivas, alta en el Core y creación de credenciales.
Notificaciones	Spring Boot	Consume eventos y envía avisos por el canal preferido, con proveedor de respaldo.
Auditoría	Spring Boot	Recibe cada acción del cliente y la guarda sin permitir cambios.
BD transaccional	Amazon Aurora PostgreSQL	Estado de transferencias, pagos, onboarding y beneficiarios. Un esquema por servicio.
Caché distribuida	Amazon ElastiCache (Redis)	Datos de clientes frecuentes, sesiones del BFF y llaves para evitar duplicados.
Bus de eventos	Amazon EventBridge + SQS	Reparte los eventos a quien los necesita, con reintentos y cola de errores.
BD de auditoría	DynamoDB + S3 Object Lock	Registro de solo inserción y archivo de largo plazo que no se puede borrar.

Tabla 2: Detalle de contenedores y responsabilidades

4.2. Servicios agregados

El enunciado parte de tres servicios. En el diseño se llaman así:

Servicio del enunciado	Nombre en el diseño
Consulta de datos básicos	Clientes
Consulta de movimientos	Movimientos
Transferencias	Transferencias

Se agrega un servicio más a los tres definidos en el enunciado. Cada uno se separa porque tiene un problema propio que no debería afectar a los demás:

- Pagos: depende de empresas de servicios externas (agua, luz, teléfono) que tienen sus propios horarios y fallas. Si una de ellas responde lento, las transferencias no se ven afectadas.
- Onboarding: maneja datos biométricos, que la ley considera sensibles. Al estar aparte, se le pueden dar permisos más estrictos que al resto.
- Notificaciones: todos los servicios necesitan avisar al cliente. Tenerlo una sola vez evita repetir esa lógica en cada uno, y sirve igual cuando BP agregue productos nuevos, como tarjetas o créditos.
- Auditoría: igual que Notificaciones, recibe las acciones de todos los servicios y se reutiliza sin cambios para productos futuros.

Además, el servicio de Clientes separa la consulta básica de la detallada. La pantalla de inicio solo necesita el nombre y los productos, que vienen del Core. El sistema complementario, que suele ser el más lento, solo se consulta cuando el usuario abre su perfil.

Decisión: microservicios por capacidad de negocio, con un BFF por canal	
Opciones evaluadas	Monolito modular; microservicios con un solo backend para ambos canales; microservicios con un BFF por canal.
Elección	Microservicios con un BFF para web y otro para móvil.
Por qué	(1) Las transferencias tienen picos en quincena y fin de mes que las consultas de perfil no tienen. Con servicios separados escalo solo lo que hace falta.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Decisión: microservicios por capacidad de negocio, con un BFF por canal	
	(2) Si Pagos falla, la consulta de movimientos sigue funcionando. Un fallo queda aislado en su servicio. (3) El BFF evita que el móvil cargue datos que solo usa la web, y deja que cada canal publique cambios a su ritmo (patrón de Sam Newman).
Lo que descarté	El monolito modular era más barato al inicio, pero con siete dominios y dos canales, tener que desplegar todo junto termina pesando más.

Tabla 3: Decisión: microservicios por capacidad de negocio

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

5. Nivel 3: Diagramas de componentes

5.1. Backend: API Gateway, microservicios e integración

Presento dos vistas: el backend con la capa de integración, donde está la mayor parte de la lógica, y la autenticación con el onboarding biométrico.

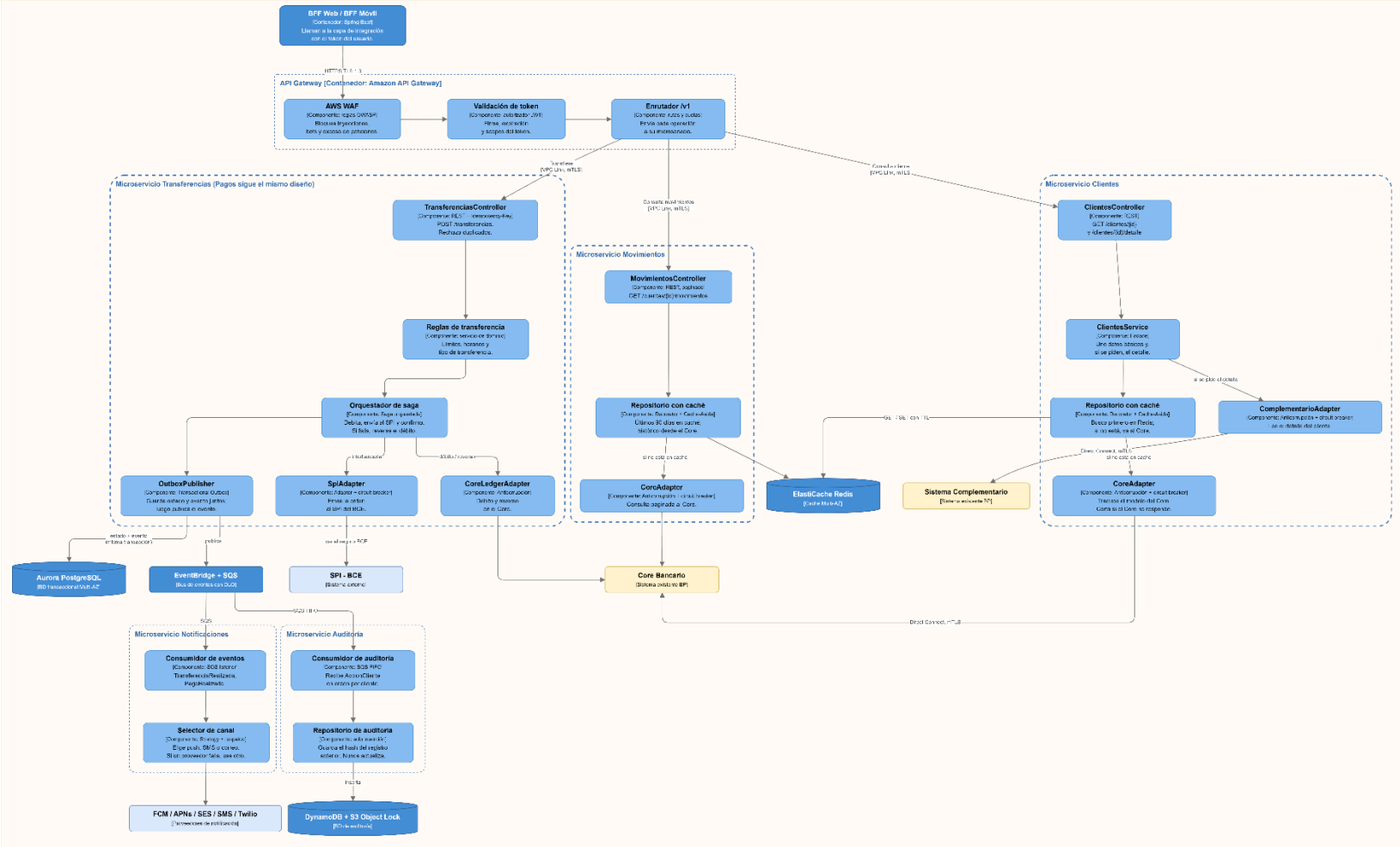


Ilustración 3: Diagramas de componentes

5.2. Patrones usados en el backend

Un patrón de diseño es una solución probada para un problema que se repite. En esta arquitectura se usa para resolver cuatro problemas concretos del backend:

- Proteger el sistema cuando algo falla afuera. El Core, el sistema complementario y el SPI son sistemas que BP no controla al 100 %. La capa anticorrupción aísla su modelo de datos, y el circuit breaker, los reintentos y los timeouts evitan que una falla externa tumbe a toda la banca.
- No perder ni duplicar dinero. Una transferencia toca dos sistemas sin una transacción común. La saga, el outbox y la llave de idempotencia garantizan que cada transferencia se procese una sola vez, que se reverse si algo falla y que siempre quede su aviso y su registro de auditoría.
- Responder rápido sin sobrecargar el Core. Cache-Aside con Decorator guarda en Redis los datos de los clientes frecuentes sin tocar la lógica de negocio.
- Poder crecer sin rehacer lo que ya funciona. El API Gateway centraliza el acceso, y publicar/suscribir permite agregar nuevos consumidores de eventos (por ejemplo, un motor antifraude) sin modificar los servicios existentes

Patrón	Dónde	Para qué
API Gateway	Entrada de la capa de integración	Concentra la validación de tokens, las cuotas y el versionado. Los microservicios no se exponen directamente.
Capa anticorrupción	Adaptadores hacia el Core, el sistema complementario y el SPI	Traduce el modelo del Core al modelo propio. Si BP cambia de Core, solo cambian los adaptadores.
Circuit breaker, reintentos y timeout	Todos los adaptadores (Resilience4j)	Si el Core responde lento, el circuito se abre y se responde con datos en caché o con un error controlado, en vez de dejar peticiones colgadas.
Cache-Aside + Decorator	Repositorios de Clientes y Movimientos	Persistencia para clientes frecuentes sin tocar la lógica de negocio (sección 10).
Saga orquestada	Transferencias y Pagos	El débito en el Core y el envío al SPI son dos sistemas sin una transacción común. Si el segundo paso falla, se reversa el primero.
Transactional Outbox	Transferencias y Pagos	El cambio de estado y el evento se guardan juntos. No hay transferencias sin aviso ni avisos de transferencias que no ocurrieron.
Llave de idempotencia	Entrada de Transferencias y Pagos	Si el cliente toca dos veces "Transferir" o la red reintenta, se procesa una sola vez.
Publicar / suscribir	EventBridge + SQS	Notificaciones y Auditoría reciben los eventos sin que Transferencias tenga que conocerlos.

Tabla 4: Patrones usados en el backend

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

5.3. Autenticación y onboarding biométrico

Componentes de la app, del proveedor de identidad y del microservicio de Onboarding que participan en el registro y en el login.

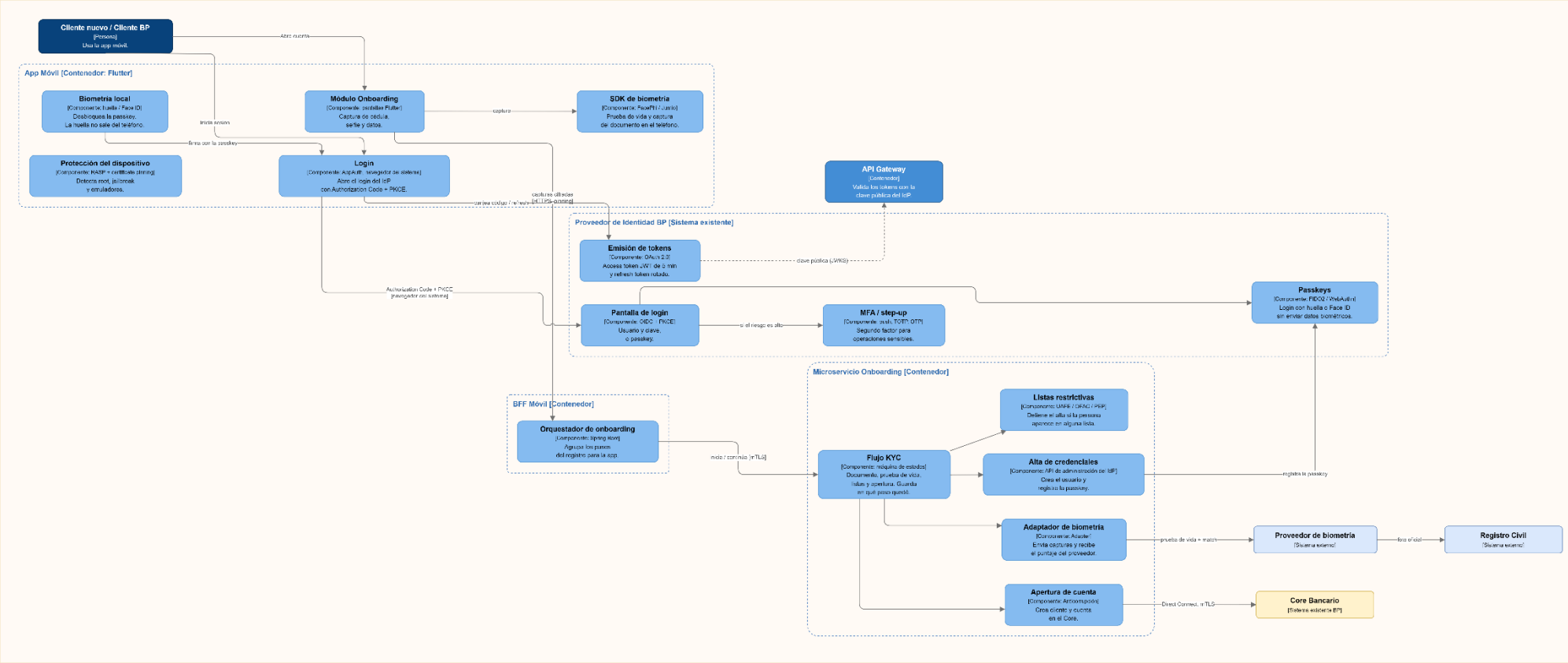


Ilustración 4: Componentes de autenticación y onboarding

6. Aplicaciones front-end

Decisión: React con TypeScript para la SPA	
Opciones evaluadas	React, Angular.
Elección	React + TypeScript.
Por qué	(1) Es el framework con más desarrolladores disponibles en la región, lo que baja el costo de contratación y de soporte. (2) Permite cargar cada sección de la banca solo cuando el usuario entra en ella, así la primera carga es liviana para clientes con conexiones lentas
Lo que descarté	Angular también era una buena opción porque trae todo integrado, pero su curva de aprendizaje es mayor.

Tabla 5: React con TypeScript para la SPA

Seguridad en la SPA: política de contenido estricta (CSP) contra inyección de scripts, ningún token en el navegador, cookie HttpOnly, Secure y SameSite=Strict, protección contra CSRF y cierre de sesión por inactividad (5 minutos, configurable).

Decisión: Flutter para la app móvil	
Opciones evaluadas	Flutter, React Native.
Elección	Flutter.
Por qué	(1) Compila a código nativo, lo que da un rendimiento estable en teléfonos de gama media, muy comunes entre los clientes. (2) Dibuja su propia interfaz, así la app se ve igual en Android e iOS y se prueba una sola interfaz. (3) Los SDK de biometría y de protección de la app que se usan en banca (por ejemplo FacePhi, Jumio, Talsec) tienen versión para Flutter.
Lo que descarté	React Native era atractivo porque reutiliza el conocimiento de React de la web, pero las librerías de seguridad nativas son más difíciles de integrar.

Tabla 6: Flutter para la app móvil

Seguridad en el móvil:

- **Certificate pinning:** la app solo confía en el certificado del BFF Móvil. Evita ataques de intermediario en redes WiFi públicas.
- **Protección de la app (RASP):** bloquea la app si detecta root, jailbreak, depuración o emuladores.
- **Almacenamiento seguro:** el refresh token se guarda cifrado en Keychain (iOS) o Keystore (Android).
- **Login con huella o Face ID:** la biometría del teléfono desbloquea una llave guardada en el chip seguro del dispositivo. La huella nunca sale del teléfono.

7. Autenticación y autorización

BP ya tiene un producto que implementa OAuth 2.0, así que no se programa la lógica de autenticación. Lo que sí hay que definir es cómo se configura y qué flujo usa cada canal.

Decisión: Authorization Code con PKCE en ambos canales	
Opciones evaluadas	Implicit; Resource Owner Password (usuario y clave enviados por la app); Authorization Code sin PKCE; Authorization Code con PKCE.
Elección	Authorization Code con PKCE (RFC 7636), como recomienda la guía de buenas prácticas de seguridad de OAuth 2.0 (RFC 9700).
Por qué	(1) La guía de seguridad de OAuth desaconseja Implicit, porque el token viaja en la URL, y Password, porque la app ve la clave del usuario y no permite MFA ni passkeys.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Decisión: Authorization Code con PKCE en ambos canales	
	(2) Con PKCE la app genera un secreto temporal y envía al IdP solo su hash. Al canjear el código presenta el secreto original. Si alguien intercepta el código, no le sirve sin ese secreto.

Tabla 7: Authorization Code con PKCE en ambos canales

7.1. Configuración por canal

Los dos canales usan el mismo flujo, Authorization Code con PKCE, pero no se configuran igual, porque el riesgo de cada uno es distinto.

En la web, todo el código corre en el navegador del cliente, donde cualquier script inyectado podría leer lo que esté guardado. Por eso el login OAuth lo hace el BFF Web y los tokens se quedan en el servidor. El navegador solo recibe una cookie de sesión que JavaScript no puede leer.

En el móvil, la app sí tiene un lugar seguro donde guardar secretos: Keychain en iOS y Keystore en Android. Por eso la app hace el login directamente, como cliente público con PKCE, y lo muestra en el navegador del sistema. Así el usuario ve la página real del IdP y la app nunca ve su clave.

En ambos canales el token dura 5 minutos y el refresh token cambia en cada uso, de modo que un token robado sirve muy poco tiempo.

Aspecto	SPA web	App móvil
Quién hace el login OAuth	El BFF Web, como cliente confidencial	La app, como cliente público con PKCE
Dónde se muestra el login	Redirección a la página del IdP	Navegador del sistema (AppAuth), nunca un WebView dentro de la app
Dónde quedan los tokens	En el BFF (Redis cifrado). El navegador solo tiene una cookie HttpOnly	Access token en memoria; refresh token en Keychain / Keystore
Duración	Access token de 5 minutos; refresh token que cambia en cada uso	Igual que la web
Scopes	cuentas.lectura, movimientos.lectura, transferencias.escritura, pagos.escritura	Igual que la web

Tabla 8: Configuración por canal

Decisión: BFF Web guarda los tokens de la SPA	
Opciones evaluadas	SPA con tokens en memoria; SPA con tokens en localStorage; BFF que guarda los tokens.
Elección	BFF que guarda los tokens y entrega una cookie HttpOnly al navegador (el patrón se conoce como token handler).
Por qué	(1) Si alguien logra inyectar un script en la página (XSS), no hay token que robar, porque el token nunca llega al navegador. (2) El BFF es un servidor y puede autenticarse ante el IdP con sus propias credenciales, algo que un código que corre en el navegador no puede hacer de forma segura.
Lo que descarté	localStorage quedó descartado porque cualquier script de la página puede leerlo.

Tabla 9: BFF Web guarda los tokens de la SPA

7.2. Métodos de autenticación

- **Usuario y clave** como método base, con política de complejidad y bloqueo por intentos fallidos.

- **Huella o Face ID con passkeys (FIDO2).** La biometría solo desbloquea una llave privada dentro del teléfono, que firma un desafío del IdP. No viaja ningún dato biométrico y es resistente al phishing. Es el método que recomiendo para el uso diario.
- **MFA para operaciones sensibles.** Consultar movimientos pide el nivel normal. Transferir a un beneficiario nuevo, superar un monto o cambiar datos de contacto pide un segundo factor: aprobación push en el teléfono enlazado, código TOTP u OTP por SMS como último recurso.
- **Evaluación de riesgo.** El IdP revisa dispositivo, IP y ubicación. Un login desde un dispositivo nuevo en otro país pide MFA aunque la clave sea correcta.

7.3. Autorización en el backend

El API Gateway valida la firma del token con la clave pública del IdP, además del emisor, la audiencia, la expiración y los scopes. Cada microservicio vuelve a verificar que la cuenta consultada pertenezca al usuario del token. Esa segunda verificación cubre el riesgo número uno de OWASP API Security: que un usuario consulte objetos de otro (BOLA). Entre servicios se usa mTLS.

8. Onboarding biométrico

El onboarding es la puerta de entrada de un cliente nuevo y también el punto preferido para los fraudes de suplantación. Por eso el flujo valida a la persona antes de crear nada en el Core.

8.1. Flujo

A. La app verifica que el teléfono no esté rooteado ni sea un emulador.

B. El cliente acepta el consentimiento para el tratamiento de sus datos biométricos.

C. El SDK del proveedor captura ambos lados de la cédula y una selfie con prueba de vida. Las capturas salen cifradas del SDK, lo que impide enviar una foto pregrabada.

D. El BFF Móvil envía las capturas al microservicio de Onboarding, que inicia el flujo KYC. El flujo guarda en qué paso quedó; si el cliente cierra la app, retoma desde ahí.

E. El proveedor de biometría compara la selfie con la foto de la cédula y con la foto oficial del Registro Civil, y devuelve los puntajes.

F. Si los puntajes superan el umbral, se revisan las listas restrictivas (UAFE, OFAC, PEP). Si hay dudas, el caso pasa a revisión manual.

G. Se crean el cliente y la cuenta en el Core.

H. Se crea el usuario en el IdP. El cliente define su clave y activa la huella o Face ID, que queda registrada como passkey. Desde ese momento entra con usuario y clave o con biometría.

I. Se publican los eventos para notificación y auditoría.

El diagrama de secuencia está en el Anexo A.

Decisión: proveedor especializado de biometría	
Opciones evaluadas	FacePhi, Jumio, Amazon Rekognition Face Liveness, desarrollo propio..
Elección	FacePhi como primera opción y Jumio como alternativa, detrás de un adaptador que permite cambiar de proveedor.
Por qué	(1) Pido certificación de detección de ataques de presentación (ISO/IEC 30107-3, evaluada por laboratorios como iBeta) para resistir máscaras, videos y deepfakes. Un desarrollo propio no llega a ese nivel en un plazo razonable. (2) FacePhi tiene presencia en banca latinoamericana e integraciones con registros civiles de la región, lo que acorta la implementación.
Lo que descarté	Rekognition es más barato por transacción, pero no valida el documento ni consulta el Registro Civil, así que habría que construir esas piezas aparte. El adaptador deja esa opción abierta si los costos lo justifican.

Tabla 10: Proveedor especializado de biometría

8.2. Tratamiento de los datos biométricos

La LOPDP clasifica los datos biométricos como sensibles. Por eso se pide consentimiento explícito antes de la captura, las imágenes se cifran con una llave dedicada al servicio de Onboarding, se guardan solo el tiempo que exige la norma y solo el área de cumplimiento puede consultarlas. Para el login diario no se usa la biometría del servidor, sino la del teléfono (passkeys), que nunca envía el rostro ni la huella.

9. Capa de integración y microservicios

Decisión: Amazon API Gateway como puerta de la capa de integración	
Opciones evaluadas	Amazon API Gateway; Kong sobre contenedores; Apigee.
Elección	Amazon API Gateway.
Por qué	(1) Es un servicio administrado: no hay servidores del gateway que operar, parchar ni escalar, y absorbe solo los picos de quincena. (2) Se integra directamente con WAF, con la validación de tokens JWT, con la red privada de los microservicios (VPC Link) y con las herramientas de monitoreo de AWS.
Lo que descarté	Kong da más plugins y sirve en varias nubes, pero agrega un clúster que operar. Apigee es fuerte en monetizar APIs, algo que BP no necesita ahora

Tabla 11: Amazon API Gateway como puerta de la capa de integración

Decisión: Amazon ECS sobre Fargate para correr los contenedores	
Opciones evaluadas	Amazon EKS (Kubernetes), ECS en Fargate, AWS Lambda.
Elección	ECS en Fargate
Por qué	(1) No hay servidores ni clúster de Kubernetes que administrar. El equipo se concentra en el negocio. (2) ECS reemplaza solo las tareas que fallan sus chequeos de salud y escala por CPU, memoria o tamaño de cola. Eso cubre el auto-healing sin piezas adicionales.
Lo que descarté	EKS sería la opción si BP ya tuviera un equipo con experiencia en Kubernetes. Lambda la descarté para los servicios principales por el tiempo de arranque en frío y por el límite de conexiones hacia el Core.

Tabla 12: Amazon ECS sobre Fargate para correr los contenedores

Decisión: Spring Boot (Java) para los BFF y los microservicios	
Opciones evaluadas	Java / Spring Boot, .NET, Node.js.
Elección	Spring Boot.
Por qué	(1) Es el ecosistema más usado en banca para integrarse con Cores (SOAP, colas de mensajes) y tiene librerías maduras de seguridad y resiliencia (Spring Security, Resilience4j). (2) Hay amplia oferta de desarrolladores Java con experiencia en banca en el país.
Lo que descarté	.NET es técnicamente equivalente y sería la elección si el equipo de BP trabaja con Microsoft.

Tabla 13: Spring Boot (Java) para los BFF y los microservicios

9.1. Comunicación entre componentes

Tramo	Protocolo	Seguridad
Canales a BFF	HTTPS / REST JSON	TLS 1.3. Cookie HttpOnly en la web; token y certificate pinning en el móvil
BFF a API Gateway	HTTPS / REST	TLS y token del usuario
API Gateway a microservicios	HTTPS por VPC Link a un balanceador interno	Red privada, mTLS y security groups
Entre microservicios	Eventos (EventBridge y SQS)	Permisos IAM por servicio y cifrado KMS

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Tramo	Protocolo	Seguridad
Nube al Core y al sistema complementario	REST o SOAP sobre Direct Connect	Enlace privado con cifrado, mTLS e IP permitidas en el firewall de BP
Transferencias al SPI	Canal definido por el BCE	Certificados del BCE y firma de mensajes
Servicios a bases de datos	Conexión nativa de cada base	TLS y credenciales rotadas en Secrets Manager

Tabla 14: Comunicación entre componentes

Decisión: comunicación asíncrona con EventBridge + SQS	
Opciones evaluadas	Amazon MSK (Kafka), SNS + SQS, EventBridge + SQS, llamadas REST directas.
Elección	EventBridge + SQS.
Por qué	(1) Desacopla: Transferencias no espera a que salga el SMS para responder, y si un consumidor se cae no se pierden eventos, porque quedan en la cola. (2) EventBridge filtra por contenido, así un servicio nuevo se suscribe solo a lo que necesita sin cambiar al que publica. (3) Se paga por uso y no hay servidores que dimensionar.
Lo que descarté	Kafka da más rendimiento y permite reprocesar eventos, pero cuesta más y hay que operarlo. Queda como evolución si el volumen crece mucho. Para la auditoría uso una cola SQS FIFO, que conserva el orden por cliente.

Tabla 15: Comunicación asíncrona con EventBridge + SQS

9.2. Transferencias interbancarias

Una transferencia a otro banco toca dos sistemas que no comparten una transacción: el Core (débito) y el SPI (envío). Lo resuelvo con una saga orquestada. El orquestador guarda cada paso en Aurora, debita, envía al SPI y confirma. Si el SPI rechaza o no responde después de los reintentos, ejecuta la compensación, que es el reverso del débito.

10. Clientes frecuentes y auditoría

10.1. Persistencia para clientes frecuentes

El enunciado pide un mecanismo de persistencia para clientes frecuentes basado en patrones de diseño. Se puede usar estos patrones que conviven juntos:

Patrón	Papel
Repository	El servicio pide los datos del cliente al repositorio y no sabe si vienen del Core o de la caché.
Decorator	Un repositorio con caché envuelve al repositorio real y le agrega la caché sin modificarlo.
Cache-Aside	Primero busca en Redis. Si no está, consulta al Core, guarda el resultado con un tiempo de vida y responde.
Invalidación por eventos	Cuando hay una transferencia o un cambio de datos, el evento borra los datos de ese cliente en la caché para que la siguiente lectura traiga datos frescos.

Tabla 16: Persistencia para clientes frecuentes

Qué es un cliente frecuente: quien entra más de cierto número de veces en una ventana de días. El umbral es configurable y se ajusta con las métricas reales de BP. Un contador en Redis lo mide. Solo a estos clientes se les guardan datos en caché, así la caché se mantiene pequeña y útil.

Qué se guarda y por cuánto tiempo: datos básicos y productos, 15 minutos; movimientos de los últimos 90 días, 5 minutos, y se borran al llegar un evento nuevo; detalle del sistema complementario, 24 horas, porque cambia poco. Los saldos nunca se toman de la caché para decidir una transferencia: el débito siempre se valida contra el Core.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Si Redis falla: el repositorio con caché detecta el error y va directo al Core. Redis corre en modo clúster en varias zonas, así que un nodo caído se reemplaza solo.

Decisión: Amazon ElastiCache (Redis) como caché distribuida	
Opciones evaluadas	Caché en memoria de cada instancia, ElastiCache Redis, DynamoDB con DAX.
Elección	ElastiCache Redis.
Por qué	(1) Una caché compartida sirve a todas las copias de un servicio. Con caché local, cada copia tendría sus propios datos y la invalidación sería inconsistente. (2) Responde en menos de un milisegundo y reduce las consultas al Core, que es el componente más caro y difícil de escalar.
Lo que descarté	DAX solo acelera DynamoDB, y los datos del cliente vienen del Core, así que no aplicaba.

Tabla 17: Amazon ElastiCache (Redis) como caché distribuida

10.2. Auditoría

La auditoría registra todas las acciones del cliente (login, consultas, transferencias, cambios de datos) de forma que nadie pueda alterarlas después, ni siquiera un administrador.

- **Captura:** cada servicio publica un evento AccionCliente con quién, qué, cuándo, desde dónde (IP y dispositivo) y el resultado. Un componente común lo hace sin repetir código en cada servicio. Los datos personales van enmascarados.
- **Transporte:** cola SQS FIFO, para conservar el orden de las acciones de cada cliente.
- **Integridad:** cada registro guarda el hash del registro anterior. Si alguien borra o cambia uno, la cadena se rompe y se detecta.
- **Almacenamiento:** DynamoDB en modo de solo inserción, con un índice por cliente y fecha para las consultas de cumplimiento.
- **Archivo:** exportación diaria a S3 con Object Lock en modo compliance. Durante el plazo de conservación que fije BP, nadie puede borrar ni modificar los archivos, ni siquiera la cuenta raíz.

Decisión: DynamoDB + S3 Object Lock para la auditoría	
Opciones evaluadas	Tabla en Aurora, Amazon OpenSearch, DynamoDB + S3 Object Lock, Amazon QLDB.
Elección	DynamoDB + S3 Object Lock.
Por qué	(1) La auditoría escribe mucho y se lee poco. DynamoDB escala la escritura sin competir con la base transaccional. (2) S3 Object Lock da inmutabilidad real durante el plazo de conservación, lo que sirve como evidencia ante el regulador.
Lo que descarté	QLDB habría sido una buena opción, pero AWS terminó su soporte en 2025. Aurora la descarté para no mezclar el volumen de auditoría con las transacciones.

Tabla 18: DynamoDB + S3 Object Lock para la auditoría

11. Notificaciones

La norma exige avisar al cliente de cada movimiento. Uso tres canales con al menos dos proveedores, así la caída de uno no deja al cliente sin aviso:

Canal	Proveedor principal	Respaldo	Uso
Push	Firebase Cloud Messaging (Android) y APNs (iOS)	SMS	Canal por defecto. No tiene costo por mensaje.
SMS	AWS End User Messaging SMS	Twilio	Transferencias, códigos OTP y cuando el push falla o el cliente no tiene la app.
Correo	Amazon SES	SendGrid	Comprobante detallado y avisos de seguridad.

Tabla 19: Notificaciones

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

El selector de canal usa el patrón Strategy: cada canal tiene la misma interfaz. Si un proveedor responde con error o no confirma la entrega a tiempo, pasa al siguiente. Agregar WhatsApp en el futuro es escribir un adaptador nuevo, sin tocar a quienes publican los eventos.

Decisión: Notificaciones como servicio propio que consume eventos	
Opciones evaluadas	Cada servicio envía sus propios avisos; plataforma externa de mensajería de marketing; servicio propio con adaptadores.
Elección	Servicio propio con adaptadores por proveedor.
Por qué	(1) Concentra en un solo lugar las plantillas, las preferencias del cliente y la evidencia de envío que pide el regulador. (2) Los datos del movimiento no salen a una plataforma de marketing externa, lo que simplifica el cumplimiento de la LOPDP y del sigilo bancario.

Tabla 20:Notificaciones como servicio propio que consume eventos

12. Infraestructura en AWS

Región principal activa con tres zonas de disponibilidad, región de recuperación en warm standby y conexión híbrida con el centro de datos de BP.

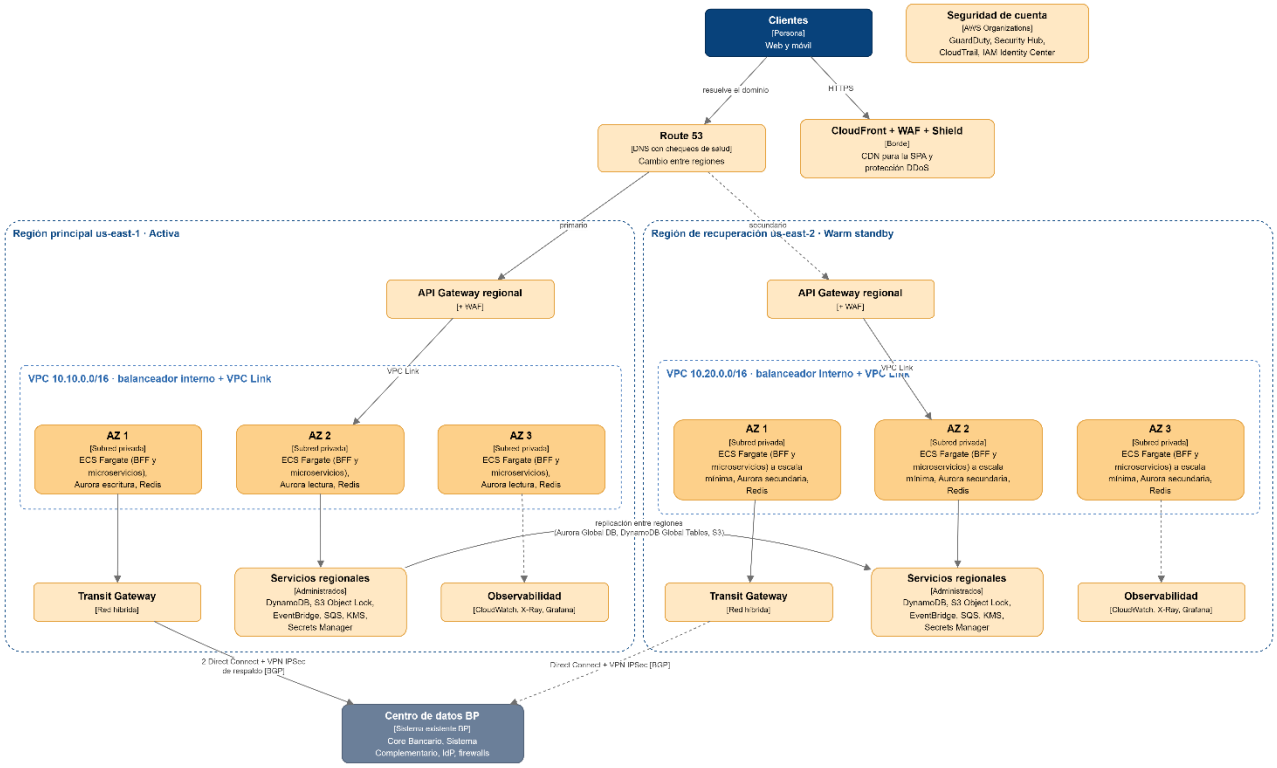


Ilustración 5: Infraestructura en AWS

12.1. Alta disponibilidad y tolerancia a fallos

Capa	Cómo resiste fallos
Borde	CloudFront y Route 53 son servicios globales. WAF y Shield frenan ataques antes de que lleguen a la región.
API Gateway y balanceador	Servicios regionales repartidos en varias zonas por diseño.
Microservicios	Mínimo dos tareas por servicio, repartidas en tres zonas. Si una zona cae, las otras absorben la carga mientras el autoescalado levanta más tareas.

Capa	Cómo resiste fallos
Aurora PostgreSQL	Una instancia de escritura y dos de lectura en zonas distintas. Si falla la de escritura, una de lectura toma su lugar automáticamente, normalmente en menos de un minuto.
ElastiCache	Modo clúster con réplicas en otra zona y cambio automático.
DynamoDB, S3, SQS, EventBridge	Replicados en varias zonas de forma nativa.
Conexión al Core	Dos circuitos Direct Connect en ubicaciones distintas y una VPN IPSec por internet como último respaldo. BGP elige la ruta disponible.
Aplicación	Circuit breaker, timeouts y reintentos con espera creciente. Si el Core no responde, se muestran los movimientos en caché con un aviso al usuario.

Tabla 21: Alta disponibilidad y tolerancia a fallos

12.2. Recuperación ante desastres

Decisión: warm standby en una segunda región	
Opciones evaluadas	Backup y restauración; pilot light; warm standby; activo-activo en dos regiones.
Elección	Warm standby en us-east-2.
Por qué	(1) Para un canal transaccional se espera recuperación en minutos. Backup y restauración tarda horas, y pilot light necesita levantar toda la capa de cómputo. (2) Warm standby mantiene la región secundaria funcionando a escala reducida. En un desastre solo hay que escalarla y redirigir el DNS.
Lo que descarté	Activo-activo da la recuperación más rápida, pero cuesta casi el doble y complica las transferencias, porque habría escrituras en dos regiones a la vez. Con warm standby el objetivo es RTO menor a 30 minutos y RPO menor a 1 minuto.

Tabla 22: Warm standby en una segunda región

- **Datos:** Aurora Global Database replica a la segunda región con un retraso típico de menos de un segundo. DynamoDB Global Tables y la replicación de S3 cubren el resto.
- **Infraestructura como código:** Terraform crea las dos regiones con los mismos módulos, así la región de recuperación no se desvía de la principal.
- **Conmutación:** Route 53 con chequeos de salud. El cambio de región lo aprueba una persona para no conmutar por una falsa alarma.
- **Pruebas:** simulacro de recuperación cada semestre.
- **Dependencia del Core:** la nube puede cambiar de región, pero si cae el centro de datos de BP manda su propio plan de continuidad. La región secundaria llega al centro de datos alterno por su propio Direct Connect.

12.3. Auto-healing

- ECS reemplaza cualquier tarea que no pase sus chequeos de salud.
- Autoescalado por CPU, memoria y tamaño de las colas, más escalado programado antes de quincena y fin de mes.
- Despliegues blue / green con regreso automático a la versión anterior si suben los errores o la latencia.
- Los mensajes que fallan varias veces van a una cola de errores (DLQ) y una alarma avisa al equipo.
- Rotación automática de credenciales en Secrets Manager.

12.4. Región y latencia

AWS no tiene región en Ecuador. Propongo us-east-1 (Virginia) como principal y us-east-2 (Ohio) como recuperación, porque tienen todos los servicios que uso y buena latencia desde Ecuador. La latencia real se confirmaría con pruebas desde la red de BP antes de decidir. La SPA se sirve desde los puntos de presencia

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

de CloudFront, y el tramo más sensible, que es el enlace con el Core, se resuelve con Direct Connect y con la caché.

13. Seguridad

Aplico la seguridad por capas y con el principio de confianza cero: ningún componente confía en otro solo por estar en la misma red.

Capa	Controles
Identidad	Authorization Code con PKCE, MFA para operaciones sensibles, passkeys, evaluación de riesgo y sesiones cortas.
Borde	AWS WAF con reglas OWASP, control de bots y límite de peticiones. Shield contra DDoS. TLS 1.3.
Aplicación	Validación de entradas, verificación de que la cuenta pertenece al usuario en cada servicio, pruebas de seguridad del código y de dependencias en cada despliegue, guías OWASP para web, móvil y APIs.
Red	Subredes privadas sin salida directa a internet, security groups mínimos, endpoints privados para los servicios de AWS, firewall para el tráfico de salida y cuentas separadas para producción, pruebas, seguridad y logs.
Datos	Cifrado en reposo con KMS y en tránsito con TLS y mTLS. Números de cuenta enmascarados en logs. Datos personales enmascarados en ambientes de prueba.
Secretos	Secrets Manager con rotación. Ningún secreto en el código.
Detección	GuardDuty, Security Hub, CloudTrail de toda la organización y envío de eventos al SIEM de BP.
Fraude	Reglas en Transferencias (frecuencia, montos atípicos, beneficiarios nuevos) y evaluación de riesgo del IdP. Integración prevista con el motor antifraude que use BP.
Accesos administrativos	Acceso con MFA y roles temporales. Nadie entra a producción salvo por un procedimiento de emergencia aprobado y registrado.

Tabla 23: Seguridad

14. Monitoreo y excelencia operativa

14.1. Observabilidad

- **Trazas:** cada petición lleva un identificador desde el BFF hasta el Core (OpenTelemetry y X-Ray). El mismo identificador viaja en los eventos y en la auditoría, así se puede seguir una transferencia de punta a punta.
- **Métricas técnicas y de negocio** en CloudWatch y Grafana: latencia, errores, transferencias por minuto, rechazos del SPI, notificaciones entregadas, tasa de aprobación del onboarding.
- **Logs** en formato JSON, sin datos personales, centralizados en CloudWatch Logs.
- **Monitoreo sintético:** un robot hace login y consulta movimientos cada minuto para detectar fallas antes que los clientes.

14.2. Objetivos de servicio y alertas

Indicador	Objetivo	Alerta
Disponibilidad de login y consultas	99,95 % mensual	Si la disponibilidad cae más rápido de lo esperado

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Indicador	Objetivo	Alerta
Latencia de consulta de movimientos (p95)	Menos de 300 ms con caché y de 1,5 s sin caché	Por encima del umbral durante 5 minutos seguidos
Transferencias exitosas	Más del 99,5 %, sin contar rechazos de negocio	Caída de la tasa o mensajes acumulados en la cola de errores
Notificación entregada	95 % en menos de 30 segundos	Retraso o falla del proveedor principal
Circuit breaker del Core	Cerrado	Cualquier apertura

Tabla 24: Objetivos de servicio y alertas

Las alertas llegan al equipo de guardia con un procedimiento de respuesta enlazado. Los cambios en producción se hacen solo por pipelines de CI/CD e infraestructura como código, con revisión por pares. Después de cada incidente se hace una revisión sin culpables. Periódicamente reviso la arquitectura contra el AWS Well-Architected Framework y su guía para servicios financieros.

15. Marco normativo y estándares

Como entidad financiera en Ecuador, BP debe cumplir la regulación local y le conviene alinearse con estándares internacionales. Esta lista no reemplaza la revisión del área legal y de cumplimiento de BP.

Norma o estándar	Qué implica en el diseño
Ley Orgánica de Protección de Datos Personales (LOPDP, 2021)	Consentimiento explícito para la biometría (dato sensible), uso limitado a la finalidad, derechos del titular, delegado de protección de datos, notificación de vulneraciones y garantías para procesar datos fuera del país.
Código Orgánico Monetario y Financiero	Sigilo bancario: los datos de cuentas y movimientos solo se muestran al titular y a quien la ley autorice. Por eso no se envían a plataformas de marketing.
Codificación de Normas de la Superintendencia de Bancos	Gestión del riesgo operativo y de la seguridad de la información, plan de continuidad con RTO y RPO definidos, gestión de proveedores de nube, seguridad en canales electrónicos y notificación de movimientos al cliente.
Ley Fintech (Ley Orgánica para el Desarrollo, Regulación y Control de los Servicios Financieros Tecnológicos)	Marco para servicios financieros digitales y futura interoperabilidad (Open Finance).
Prevención de lavado de activos (UAFE)	Conocimiento del cliente en el onboarding, listas restrictivas y reporte de operaciones inusuales.
Reglas del Sistema de Pagos Interbancarios (BCE)	Formato, horarios, certificados y conciliación de las transferencias a otros bancos.
ISO/IEC 27001 e ISO 22301	Gestión de la seguridad de la información y de la continuidad del negocio, base del plan de recuperación.
PCI DSS	Aplica si en el futuro la banca muestra o procesa datos de tarjetas. La red ya está segmentada para aislar ese alcance.
OWASP (Top 10, API Security, guías móviles)	Requisitos de seguridad para web, móvil y APIs, y base de las pruebas de penetración.
ISO/IEC 30107-3	Detección de ataques de presentación en la biometría facial.

Tabla 25: Marco normativo y estándares

16. Manejo de costos

El enunciado no da volúmenes de uso, así que no presento un monto mensual: cualquier cifra sería una suposición sin respaldo. Explico qué determina el costo de cada componente, cómo lo controla el diseño y cómo se calcularía el presupuesto con los datos reales.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Componente	Qué determina el costo	Cómo lo controla el diseño
ECS Fargate	CPU y memoria por hora de cada tarea	Autoescalado, menos tareas fuera de horas pico y planes de ahorro para la carga base
Aurora PostgreSQL	Tamaño y número de instancias, almacenamiento	La caché absorbe las lecturas repetidas
ElastiCache	Tamaño y número de nodos	Solo se guardan datos de clientes frecuentes
API Gateway y CloudFront	Número de llamadas y datos transferidos	Los BFF agrupan llamadas; la SPA se sirve desde la caché de CloudFront
DynamoDB y S3 (auditoría)	Escrituras y almacenamiento	Pago por uso y paso a almacenamiento de archivo (Glacier) después de 90 días
Direct Connect	Capacidad del puerto y datos que salen de AWS	La capacidad se elige con el tráfico medido hacia el Core
Región de recuperación	Lo mismo que la principal, a escala mínima	Warm standby en lugar de activo-activo
Fuera de AWS	Licencias del IdP, biometría por validación, SMS por mensaje	Push primero; SMS solo como respaldo y para OTP

Tabla 26: Manejo de costos

Para dimensionar con datos reales:

A. Pedir a BP sus cifras: clientes digitales, usuarios activos por día y por mes, transacciones por tipo y picos históricos (quincena, fin de mes).
B. Convertirlas en peticiones por segundo por servicio.
C. Hacer una prueba de carga en preproducción para medir cuántas peticiones soporta cada tarea y cuántas llegan al Core.
D. Con esos resultados, fijar mínimos y máximos de autoescalado, tamaños de base de datos y caché, y capacidad de Direct Connect.
E. Calcular el presupuesto en AWS Pricing Calculator y revisarlo cada mes contra el gasto real con etiquetas por servicio y alertas de presupuesto.

17. Registro de decisiones de arquitectura

#	Tema	Elección	Alternativas	Razones principales
1	Estilo	Microservicios + BFF por canal	Monolito modular, backend único	Escalado por servicio, fallos aislados, canales independientes
2	SPA	React + TypeScript	Angular	Oferta de talento, carga liviana
3	Móvil	Flutter	React Native	Rendimiento nativo, una sola interfaz, SDK de seguridad disponibles
4	Flujo OAuth	Authorization Code + PKCE	Implicit, Password, sin PKCE	RFC 9700, código interceptado no sirve
5	Tokens en web	Guardados en el BFF	Memoria o localStorage	Fuera del alcance de un XSS
6	Login biométrico	Passkeys (FIDO2)	Biometría en servidor, solo clave	La biometría no sale del teléfono, resiste phishing
7	Onboarding	FacePhi (Jumio alternativa)	Rekognition, desarrollo propio	Certificación anti-suplantación, integración con Registro Civil
8	API Gateway	Amazon API Gateway	Kong, Apigee	Administrado, integrado con WAF y red privada

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

#	Tema	Elección	Alternativas	Razones principales
9	Cómputo	ECS Fargate	EKS, Lambda	Sin servidores que operar, auto-healing nativo
10	Backend	Spring Boot	.NET, Node.js	Ecosistema bancario, librerías de resiliencia
11	Mensajería	EventBridge + SQS	Kafka, SNS + SQS, REST	Desacople, filtrado por contenido, pago por uso
12	Transferencias	Saga orquestada + outbox + idempotencia	Saga coreografiada	Estado trazable, sin duplicados, eventos confiables
13	Clientes frecuentes	Repository + Decorator + Cache-Aside en Redis	Caché local, DAX	Caché compartida, menos carga al Core
14	Base transaccional	Aurora PostgreSQL	RDS PostgreSQL, DynamoDB	Cambio rápido ante fallas, réplica a otra región
15	Auditoría	DynamoDB + S3 Object Lock + hash encadenado	Aurora, OpenSearch, QLDB	Escritura masiva, inmutable, QLDB sin soporte
16	Notificaciones	Servicio propio, 3 canales con respaldo	Envío por servicio, SaaS de marketing	Evidencia centralizada, sigilo bancario
17	DR	Warm standby en us-east-2	Backup, pilot light, activo-activo	RTO < 30 min con costo moderado
18	Conectividad	2 Direct Connect + VPN IPSec	Solo VPN	Latencia estable, ancho de banda dedicado, redundancia

Tabla 27: Registro de decisiones de arquitectura

17.1. Riesgos y mitigaciones

Riesgo	Mitigación
El Core no soporta los picos de consultas	Caché para clientes frecuentes, límite de conexiones por servicio y pruebas de carga contra el Core antes de salir a producción.
Restricciones para procesar datos fuera del país	Validarlo temprano con la Superintendencia de Bancos. El diseño permite mantener los datos del cliente en el centro de datos de BP y procesar en la nube solo lo transitorio si fuera necesario.
Fraude por suplantación en el onboarding	Prueba de vida certificada, capturas cifradas desde el SDK, protección de la app, listas restrictivas y revisión manual de casos dudosos.
Caída de un proveedor de notificación	Al menos dos proveedores por canal y cambio automático.
Complejidad de operar microservicios	Plantillas comunes para todos los servicios, monitoreo desde el primer día y servicios administrados donde sea posible.

Tabla 28: Riesgos y mitigaciones

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.

Anexo A. Diagramas de secuencia

A.1 Login en la SPA con Authorization Code + PKCE

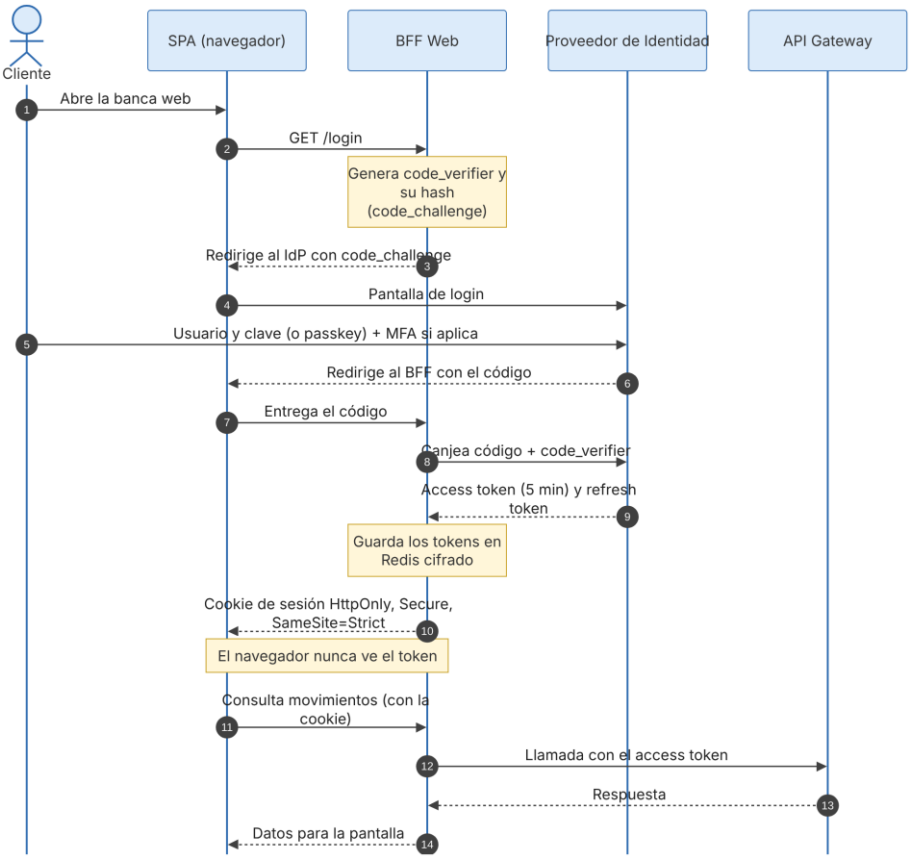


Ilustración 6: Login en la SPA

A.2 Onboarding con reconocimiento facial

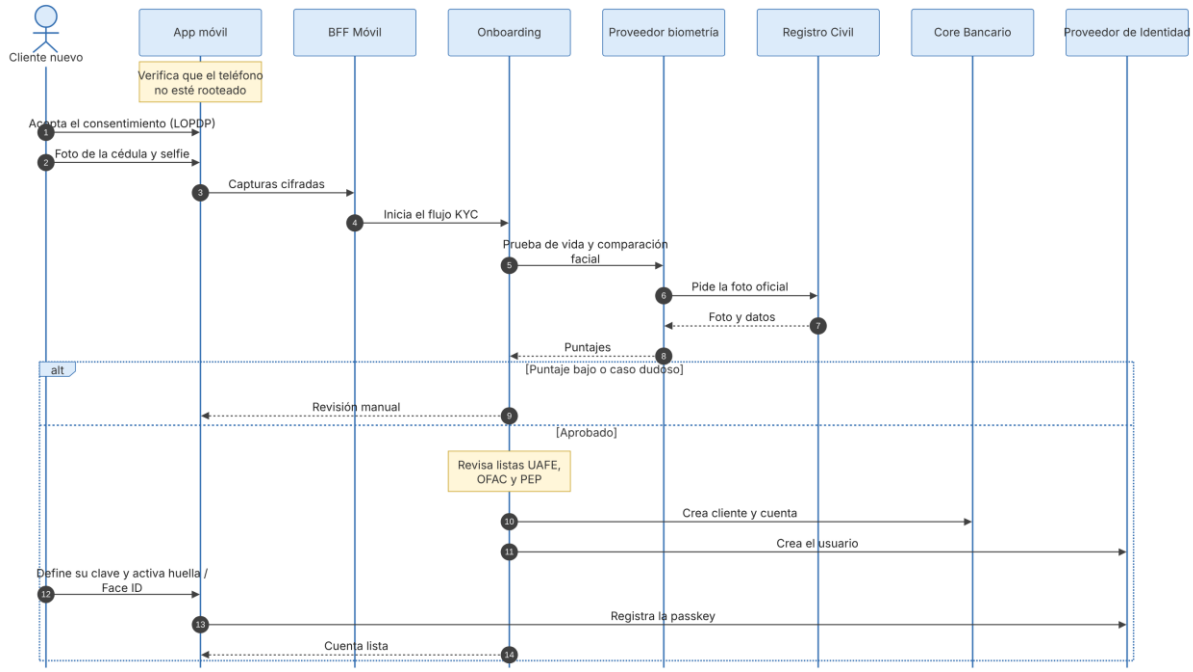


Ilustración 7: Onboarding biométrico

A.3 Transferencia interbancaria

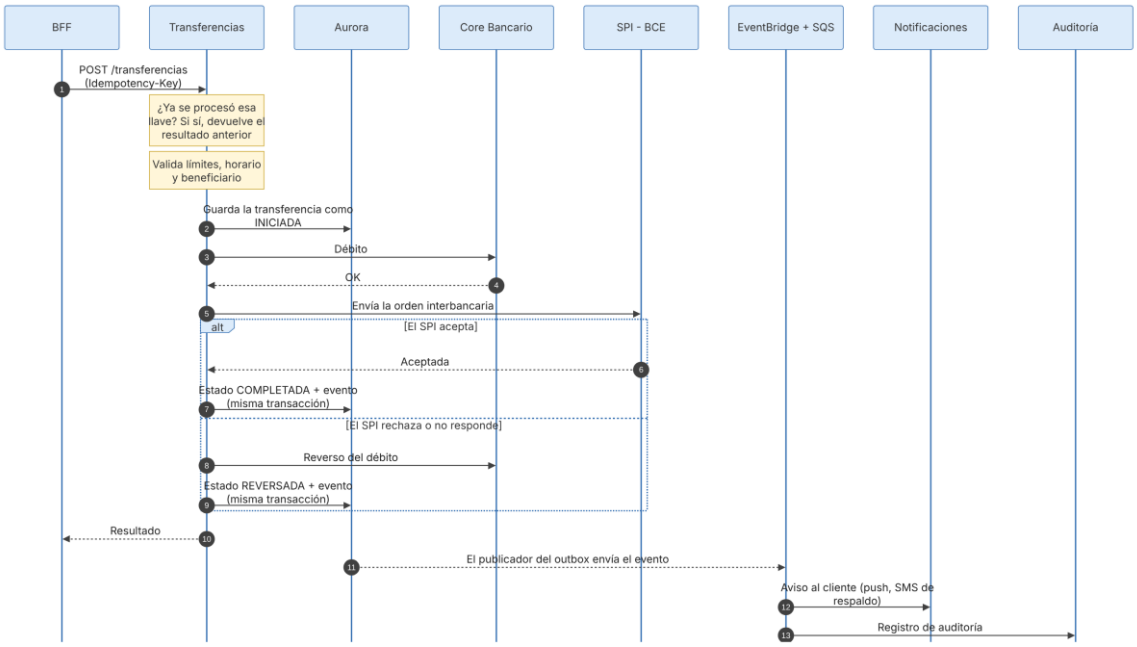


Ilustración 8: Transferencia interbancaria con saga y outbox

Anexo B. Fuentes de los diagramas

Los diagramas de contexto y de contenedores están hechos en draw.io. Todo está en el repositorio público del ejercicio, en la carpeta diagramas/.

Referencias

- Simon Brown, The C4 model for visualising software architecture, c4model.com.
- IETF RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 8252 (OAuth para apps nativas), RFC 9700 (buenas prácticas de seguridad de OAuth 2.0).
- Sam Newman, Building Microservices, 2.ª ed. (BFF, sagas, división por dominio).
- Chris Richardson, Microservices Patterns (Saga, Transactional Outbox, API Gateway).
- Eric Evans, Domain-Driven Design (capa anticorrupción).
- AWS Well-Architected Framework y Financial Services Industry Lens; AWS, Disaster Recovery of Workloads on AWS.
- OWASP Top 10, API Security Top 10 y MASVS.
- Asamblea Nacional del Ecuador, Ley Orgánica de Protección de Datos Personales (2021).
- Superintendencia de Bancos del Ecuador, Codificación de Normas.

Confidencial · Uso exclusivo para el proceso de evaluación de Devsu. Prohibida su distribución sin autorización del autor.